像素操作

在这之前我们多次提到 `canvas` 获取像素数据的API： `getImageData()` ，现在就来更深入地了解它吧。

1） `ImageData` 对象

我们通过 `getImageData` 可以拿到一个 `ImageData` 对象，这个对象也就是我们所说的像素数据，下面示例中我们可以通过鼠标移动获取鼠标所在位置**1个像素格**的像素数据：

https://codepen.io/huanjinliu/embed/oNQqVGj?editors=1011

```
ImageData
  data: Uint8ClampedArray(4) [255, 255, 255, 255]
  colorSpace: "srgb"
  height: 1
  width: 1
```

`data` 存放当前提取的像素数据，是一个8位无符号整型数组。

`colorSpace: srgb` 指使用RGBA色彩空间，每个像素点使用四个数值来表示其色值，分别是:

`R-red` 、 `G-green` 、 `B-blue` 、 `A-opacity`

(注意：这里透明度不是我们常见的[0, 1]，而是[0, 255])

`width` 、 `height` 毫无疑问就是当前提取像素数据的宽高。

1）通过位置计算像素在data中的开始索引（即红色通道值所在索引）：

`let index = row * widrth * 4 + col * 4`

绿色通道值、蓝色通道值及透明度，都紧随其后，通过 `[index + 1]` 、 `[index + 2]` 、 `[index + 3]` 即可计算得出。

2）涉及尺寸调整，请创建新的 `ImageData` 对象

这里注意的是， `getImageData` 获取的像素数据中，所有字段都是只读的，以下操作都是无效的：

```
- imageData.width = 100;
- imageData.data = new Uint8ClampedArray(10);
...
// 但是可以直接改imageData.data内的像素值
// 如: imageData.data[0] = 0;
```

所以如果你打算直接通过修改宽高字段来裁切 `ImageData` 对象，是做不到的，只能另外 `new ImageData(width, height)` 创建一个新的 `ImageData` 对象，然后将数据写进去。

3） `putImageData(ImageData, x, y)` 将 `ImageData` 像素数据写入画布

比起 `drawImage` ， `putImageData` 的效率要慢得多，请注意使用。

思考：像素处理除了用于图像相关的处理，你有想到其他的用途吗？【请观察最上方的DEMO】

解答：

由于画布中绘制的图案并不像文档中的标签节点那样可以设置自己的点击事件，所以我们如果要实现画布元素的点击事件，也就是知道当前点击的是什么图形，那就得自行实现一些方法，其中最简单粗暴且十分强大的就是“取色值法”，它可以匹配不规则图形的点击事件。

具体操作也非常容易理解，就是需要另外一个游离画布单独绘制该元素，然后在对主画布的点击事件进行监听时，判断在该游离画布相同点击位置上是否存在非透明节点，如果存在即可快速判断该元素被鼠标点击了。

当然它也有缺点，那就是 `getImageData` 比较慢且多创建一个游离画布同步绘制本身消耗也不小。

2） `getImageData` 最佳实践

`getImageData` 是一个相对消耗性能的操作，因为它需要从GPU中读取数据，然后将这些数据复制到内存空间中，这个过程涉及到 CPU 和 GPU 之间的数据传输，是一个相对较慢的操作。另外，每次调用都会创建一个新的 `ImageData` 对象，也会增加内存的使用。

所以使用时，宁可一次先拿到所有需要读取的像素数据，也不要分开一次次读取：

```
// BAD
for (let row = 0; row < canvas.height; row++) {
  for (let col = 0; col < canvas.width; col++) {
    const imageData = ctx.getImageData(row, col, 1, 1);
    let r = imageData.data[0];
    let g = imageData.data[1];
    let b = imageData.data[2];
```

```
      let a = imageData.data[3];
    }
  }

  // GOOD
  const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
  for (let row = 0; row < canvas.height; row++) {
    for (let col = 0; col < canvas.width; col++) {
      const index = row * canvas.height * 4 + col * 4;
      let r = imageData.data[index];
      let g = imageData.data[index + 1];
      let b = imageData.data[index + 2];
      let a = imageData.data[index + 3];
    }
  }

  // BETTER ! 像素操作是很耗时的，所以编码上要更加注重性能问题
  const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
  for (let index = 0; index < imageData.data; index+=4) {
    let r = imageData.data[index];
    let g = imageData.data[index + 1];
    let b = imageData.data[index + 2];
    let a = imageData.data[index + 3];
  }
```

> `canvas.getContext('2d', { willReadFrequently: true });`
>
> `willReadFrequently` 这个属性表示你将计划频繁地读取 canvas 的像素数据，浏览器会对canvas的内部进行优化以提高读取像素数据的性能。但是会降低绘制性能，而且是个实验性选项，兼容性相当差（新版谷歌支持）。